

Word2Vec

단어 벡터화는 단어를 벡터로 표현하는 방법으로, 크게 희소 표현과 밀집 표현으로 나뉜다. TF-IDF 와 같은 희소 표현은 단어 사전 크기에 따라 벡터 크기가 커져 공간이 낭비되고 단어 간 유사성을 반영하지 못하는 한계가 있었다. 이러한 문제를 해결하기 위해 Word2Vec 과 같은 밀집 표현(Distributed Representation)이 등장했으며, 이는 "같은 문맥에서 자주 나타나는 단어는 유사한 의미를 가진다"는 분포 가설을 기반으로 한다. 그 결과, 학습을 통해 단어의 의미를 벡터 공간에 매핑함으로써 단어 간의 의미 비교가 가능해졌다.

Word2Vec 은 밀집 표현을 학습하기 위해 CBOW 와 Skip-gram 이라는 두 가지 방법을 사용한다. CBOW 는 주변 단어를 이용해 중심 단어를 예측하는 방식이며, 반대로 Skip-gram 은 중심 단어를 입력받아 주변 단어를 예측하는 모델이다. Skip-gram 은 하나의 중심 단어에서 여러 학습 데이터를 추출할 수 있어 CBOW 보다 더 많은 데이터셋을 활용할 수 있으며, 일반적으로 더 뛰어난 성능을 보인다.

Word2Vec 모델은 단어 사전이 커질수록 소프트맥스 연산으로 인해 학습 속도가 느려지는 단점이 있었다. 이 속도 문제를 완화하기 위해 계층적 소프트맥스(Hierarchical Softmax)와 네거티브 샘플링(Negative Sampling) 기법이 적용되었다. 계층적 소프트맥스는 출력층을 이진 트리 구조로 표현하여, 전체 단어에 대해 연산하던 시간 복잡도를 로그 복잡도로 크게 줄였다. 네거티브 샘플링은 전체 단어 대신, 정답 단어와 함께 원도에 등장하지 않는 일부 오답 단어(네거티브 샘플) n 개만 추출하여 연산을 수행한다. 이를 통해 다중 분류 문제를 이진 분류 문제로 변환하여 모델을 효율적으로 학습시킨다.

모델 실습 시, 파이토치(PyTorch)의 `nn.Embedding` 과 `nn.Linear` 를 사용해 Skip-gram 모델을 직접 구현할 수 있으며, 학습된 임베딩 행렬 간의 코사인 유사도를 계산하여 단어의 의미적 유사성을 확인할 수 있다. 혹은 Gensim 라이브러리를 활용하면, 내부적으로 병렬 처리와 네거티브 샘플링 등이 적용되어 있어 파이토치로 직접 구현하는 것보다 훨씬 빠른 속도로 모델을 학습시킬 수 있다.

fastText

Word2Vec 은 단어 자체를 최소 단위로 보았기에 단어의 형태학적 특징을 반영하지 못하는 한계가 있었다. 특히 한국어와 같은 교착어는 어근, 접사 등의 구조적 특징을 학습하기 어려워 단어 사전에 없는 단어(OOV) 문제를 야기했다. fastText 는 이러한 한계를 극복하기 위해 N-gram 을 사용하여 단어를 하위 단어(Subword) 단위로 분해하고 벡터화하는 방법을 도입했다.

fastText 는 단어의 하위 단어 집합을 생성하고, 각 하위 단어의 임베딩 벡터를 모두 합산하여 해당 단어의 최종 임베딩 벡터를 계산한다. 이 방식을 통해 말뭉치에 등장하지 않은 OOV 단어라도, '사랑해요'가 '사랑'이나 '해요' 같은 하위 단어로 분해되듯이, 유사한 하위 단어 정보를 활용하여 임베딩을 연산할 수 있게 되었다. 모델 실습에서 Gensim 의 FastText 클래스를 사용하면, 단어 사전에 없는 토큰에 대해서도 의미적으로 유사한 단어를 효과적으로 추출할 수 있다.

순환 신경망 (RNN)

순환 신경망(Recurrent Neural Network, RNN)은 자연어와 같이 순서가 있는 연속적인 데이터를 처리하는 데 적합한 구조를 가진 신경망이다. 자연어는 문장 안에서 단어들이 순서대로 등장하며 이전 단어의 영향을 받아 의미를 형성하기 때문에 연속형 데이터의 특징을 갖는다. RNN 은 각 시점마다 이전 시점의 은닉 상태를 입력으로 받아 현재 시점의 은닉 상태를 계산하는 재귀적 특징을 통해 과거의 상태를 기억한다. RNN 은 다대일(감성 분류), 다대다(번역) 등 다양한 구조로 설계될 수 있다.

기존의 순환 신경망은 시간적으로 먼 과거의 정보를 잘 기억하지 못하는 장기 의존성 문제(Long-term dependencies)가 있었으며, 역전파 과정에서 기울기 소실 문제가 발생할 가능성이 있었다. 장단기 메모리(Long Short-Term Memory, LSTM)는 이러한 문제를 해결하기 위해 메모리 셀과 3 개의 게이트(망각, 입력, 출력)라는 구조를 도입한 모델이다. 이 게이트들은 시그모이드 함수를 통해 0 에서 1 사이의 값을 출력하며, 이를 이용해 이전 정보 중 어떤 것을 삭제할지(망각 게이트), 새로운 정보 중 어떤 것을 추가할지(입력 게이트), 셀 상태 중 어떤 것을 출력할지(출력 게이트)를 동적으로 제어한다.

모델 실습에서는 RNN 이나 LSTM 을 활용해 네이버 영화 리뷰 데이터세트의 긍/부정을 분류하는 모델을 구현할 수 있다. 이때 임베딩 계층은 무작위 값으로 초기화할 수도 있지만, Word2Vec 등으로 미리 학습된 사전 학습 임베딩 벡터를 초깃값으로 적용하여 모델 성능을 개선할 수도 있다.

합성곱 신경망 (CNN)

합성곱 신경망(Convolutional Neural Network, CNN)은 본래 이미지의 지역적인 특징을 추출하는 데 특화된 구조였지만, RNN 이 가진 병렬 처리의 어려움과 긴 데이터 처리 속도 문제를 해결할 수 있어 자연어 처리(NLP) 분야에도 사용되기 시작했다.

텍스트 데이터는 이미지와 달리 2 차원 공간 정보가 의미를 가지지 않으므로, 텍스트 임베딩 벡터를 처리하기 위해 1차원 합성곱(1D Convolution)을 적용한다. 1차원 합성곱은 필터가 수직 방향(단어 시퀀스 방향)으로만 이동하며, 이는 N-gram 과 유사하게 인접한 N 개의 토큰 정보를 하나의 특징으로 추출하는 효과를 낸다.

CNN 은 여러 계층으로 구성된다. 합성곱 계층은 입력 데이터에 필터를 이용해 합성곱 연산을 수행하는 계층이다. 이때 출력 크기가 작아지는 것을 방지하기 위해 입력 가장자리에 0 을 덧붙이는 패딩을 사용하고, 필터가 한 번에 움직이는 크기인 간격을 조절하여 출력 데이터 크기를 제어한다.

합성곱 연산을 거친 특징 맵은 비선형성을 추가하기 위해 ReLU 와 같은 활성화 함수를 통과하여 활성화 맵이 된다. 이후 풀링 계층은 특징 맵의 크기를 줄여 연산량을 감소시키고 정보를 압축하는 역할을 한다. 주로 필터 내 가장 큰 값만 선택하는 최대값 풀링(Max Pooling)이 사용된다. 마지막으로, 추출된 모든 특징 맵은 평탄화(Flatten) 작업을 통해 1 차원 벡터로 변경되고, 완전 연결 계층(Fully Connected Layer)에 전달되어 최종적인 분류 작업을 수행한다.

모델 실습에서는 1 차원 합성곱 기반의 문장 분류 모델을 정의할 수 있다. 이때, 그림 6.31 처럼 크기가 다른 여러 개의 합성곱 필터(예: [3, 3, 4, 4, 5, 5])를 병렬로 구성하여, 각 필터가 추출한 특징 벡터들을 torch.cat 을 이용해 하나로 이어 붙인 뒤 분류기(Classifier)에 전달한다.